

Protocolos de comunicacion

Jaime Nazar Anchorena

Contents

Unidad 10: SSH	5
VPN	5
Local port forwarding	5
Remote port forwarding	5
Dynamic Port Forwarding	6
Bastions	6
Notas practicas	6
Unidad 11: Socket bloqueantes	6
Berkeley	7
Sockets	8
Creacion de un socket	8
Socket I/O	8
Formato de direcciones	9
Resolucion de nombres	9
Diseno de servidores	10
Algoritmo para servidor iterativo no orientado a conexion	10
Algoritmo para servidor iterativo orientado a conexion	10
Unidad 1: Introduccion	10
Modelo OSI - TCP/IP	11
Capa de enlaces	12
Capa de ruteo	12
Capa de transporte: Puestos	13
Capa de aplicacion: soporte a aplicaciones	13
Serializacion	13
Unidad 2: HTTP	13
HTTP 0.9(1991)	13
HTTP 1.0(1996)	13
HTTP 1.1(1999)	13
HTTP/2	14
HTTP/3	14
Recursos	14
URI(RFC 3986 3.3)	14

URL	14
URN	14
Request menos utilizados	15
Respuestas	15
Tipos de informacion	15
GET vs POST	15
Headers HTTP	16
Tipos	16
Headers generales	16
Headers de solicitud	16
Headers de respuesta	17
Headers de contenido	17
Proxy servers	17
Proxy reverso	18
Cookies	18
Curl	18
Resolucion de nombres - DNS	19
Como se consiguen los nombres	20
Cache	20
Configuracion de servidor DNS	20
Registros DNS	21
Formato RR	21
Split-horizon	21
DNS dinamico	21
DNS spoofing	21
Metodo: Enviar respuesta sin recibir pregunta	22
Metodo: DNS sniffer	22
Metodo: DNS cache poisoning	22
Notas practica	22
Unidad 4: Correo electronico	23
Envio de datos binarios	23
Como mandar mail con netcat	23
Protocolos de entrega final al usuario	24
Encoding: Base64	25
SPAM	25
Técnicas para autenticar mails	25
SPF	25
DKIM	25
Notas practica	25
Unidad 5: TLS/SSL	25
SSL	25
Claves asimetricas	26
Tipos de SSL handshake	26
TLS	27

DNS over TLS(DoT)	27
DNS over HTTPS(DoH)	27
HTTPs y proxy HTTP	27
Intercepcion HTTPS con MITM	27
Deteccion	27
Limitaciones	27
Unidad 6: Protocolos de transporte	27
Multiplexacion	28
Demultiplexacion sin conexion	28
UDP	28
Demultiplexacion orientado a conexion	28
TCP	29
Repaso conexion confiable	29
Control de flujo	30
Control de congestion	30
NAPT (aka NAT)	30
SNAT	30
DNAT	31
Firewall	31
Notas practica	31
Telnet	31
SCTP	31
Inicio protegido	32
Fin	32
Porque no se usa?	32
Unidad 7: Capa de red	32
Direcciones en tabla de ruteo	33
Subredes(VLSM)	34
Motivos para crear subredes	35
Superred(CIDR)	35
Paquete IPv4(datagrama)	35
Asignacion de direcciones IP	35
DHCP	35
Network Address Translation(NAT)	36
DNAT(Destination NAT)	36
ICMP	36
IPv6	37
Notacion abreviada	37
Tipos	37
Clases	37
Tipos(de otra forma)	38
Interface ID	38
Ipv\$-Mapped IPv6	38
NAT64	38

Tunneling	38
Seguridad	38
Firewalls	39
IP Tables	39
Notas practica	40
Unidad 8: Protocolos de ruteo	41
Ruteo minimo	41
Ruteo estatico	41
Ruteo dinamico	41
Algoritmos de ruteo descentralizado	41
Algoritmos de ruteo globales	41
Sistemas autonomos	42
RIP	42
OSPF	42
Unidad 9: Capa de enlace	42
Point to point	42
Protocolos	42
Broadcast	43
Protocolos	43
Ethernet	44
Topologia bus	44
Topologia estrella	44
MAC address	44
Preambulo	44
ARP	44
ARP spoofing	44
Notas practica	45

Unidad 10: SSH

VPN

Un tunel es una encapsulacion de un protocolo de la misma o menor capa, HTTP por IP no es un tunel. Un tunel es cuando protocolo lleva un paquete de igual o menos nivel.

Una de las razones para usar VPN es para administrar algun host desde otra red, no es la unica manera, pero es la mas segura. La gateway de la red del VPN sera 0.0.0.0 pues esta en mi misma red. Va todo encriptado, de manera que nadie mas que este en el medio puede ver lo que se esta mandando salvo la IP destino y de origen. Si quiero que todo mi trafico vaya por el VPN hay que agregar otra entrada con el gateway de la otra red, aca entran en juego las prioridades de la tabla de ruteo.

net	mask	gateway	link	prio
192.168.1.3	255.255.255.255	0.0.0.0	lo	
0.0.0.0	0.0.0.0	192.168.0.1	eth0	20
192.168.1.0	255.255.255.0	0.0.0.0	eth0	
127.0.0.0	255.0.0.0	0.0.0.0	eth0	
0.0.0.0	0.0.0.0	10.0.0.1	vpn0	10
10.0.0.2	255.255.255.255	0.0.0.0	lo	
10.0.0.0	255.255.0.0	0.0.0.0	vpn0	

El protocolo SSH ofrece:

- Autenticacion
- Encriptacion
- Integridad(detecta si el mensaje fue alterado)

Cuando te conectas por primera vez te pide verificar si confias en el certificado del servidor porque no te puede garantizar que sea el servidor que buscas. No ocurre con HTTPS pues se usan certificados firmados que garantizan su autenticidad.

`LoginGraceTime` indica cuanto tiempo espera a que el usuario se autentique antes de cortar la conexion. `StrictMode` se fija que el usuario no haya publicado su configuracion, se puede no permitir.

SSH es un tunel pues puede llevar datos de otra aplicacion, aprovechando la conexion segura. Esto se conoce como **tunneling**. Se podria usar de la siguiente manera:

```
ssh -L8080:webserver:80 SSHserver
```

Local port forwarding

Se puede habilitar para que no solo escuche a localhost, de manera que otras personas se pueden hacer pedidos de otros servicios y estos van a pasar por el servidor SSH.

Remote port forwarding

Misma idea, pero ahora el tunel pasivo esta en el lado del servidor SSH. Se usa cuando la aplicacion esta del lado del cliente. Sin embargo, esto no esta bueno a nivel seguridad, pues cualquiera se va a poder conectar

a la aplicacion, solo se debe usar para casos extremos.

Dynamic Port Forwarding

Solo hay que especificar el servidor SSH, solo funciona con un proxy especial. Tiene la ventaja que todos los servicios se van por el tunel y no solo uno especificado:

```
ssh -D 9090 pampero.itba.edu.ar
```

Donde 9090 es el servidor proxy. Por ejemplo, abrir Google Chrome desde el proxy:

```
google-chrome --proxy-server='sock5://localhost:9090'
```

Como AWS tiene un rango de IPs conocidas, hay sitios que detectan el tunel

Tiene sentido usar `PasswordAuthentication` no cuando se va a usar solo para tuneles, asi no hay que ingresar con password.

Si no tengo usar una IP fija puedo usar pampero y conectarme por ssh. De esa manera puedo conectarme a AWS desde pampero y luego a desde mi casa a pampero.

Bastions

Donde hay un hosts con acceso a internet y a una red interna, si pasa el trafico por ahi las claves no deben estar ahi por cuestiones de seguridad. Entonces, como saben los hosts en la red interna que el bastion tiene las claves privadas de el que se conecta desde afuera? Se usa algo llamado **ssh-agent**, el cual crea un tunel que se usa en el bastion, de manera que se pueda completar el challenge sin que la clave privada del host exterior salga de el.

Para crear esto:

```
eval `ssh-agent`
```

Notas practicas

- No hay manera de protegerse de un man-in-the-middle en la primer conexion, tenemos que confiar que el que esta del otro lado es quien dice ser.
- **TCPKeepAlive**, desde el lado del cliente, permite mantener la conexion con vida.
- Root login debe estar con **no**, **prohibit-password** es poco seguro.
- La principal diferencia con VPN es la capa en la que actua, por ejemplo, SSH es a nivel de aplicacio:wqn y VPN a nivel de red.

Unidad 11: Socket bloqueantes

En un servidor TCP hay dos tipos de sockets:

- Pasivo
- Activo

En UDP solo hay uno pues no esta la necesidad de mantener un estado. Las aplicaciones de red tienen:

- Comunicacion entre procesos(En un mismo host o en diferentes hosts)

- **Proceso cliente:** Inicia la conexión
- **Proceso servidor:** Espera conexiones
- Se debe poder identificar los procesos
- Cada host se identifica por IP
- Muchos procesos corriendo en un host
- Para identificar procesos se usa el número de puerto

Si están en diferentes hosts, hay que identificar ambos hosts. Para TCP cada uno envía y recibe, no hay distinción entre cliente y servidor. El puerto del cliente es elegido al azar, mientras que para el servidor se tiene que seguir convenciones.

El protocolo a nivel de aplicación debe definir:

- Tipos de mensajes
- Sintaxis de los mensajes
- Semántica de los mensajes
- Reacción frente a errores

Es más fácil programar un protocolo que es binario pues no hay que parsear como se tendría que hacer con texto plano

El estándar TCP/IP sugiere que las API contemplen las siguientes operaciones básicas:

- Reservar recursos locales para la comunicación
- Especificar endpoints origen y destino
- Iniciar conexión(desde cliente)
- Enviar datagrama(desde cliente)
- Esperar una conexión(desde servidor)
- Enviar y recibir datos
- Poder determinar cuando arriba información
- Generar datos urgentes
- Manejar arribo de datos urgentes

Algunas API:

- socket API(Berkeley)
- Windows Sockets
- Transport Layer Interface TLI

Nosotros vamos a usar socket API

Berkeley

Diseñado en los 80 para UNIX, utiliza system calls ya existentes cuando sea posible. Vamos a tener que especificar que el socket es para TCP/IP y que tipo de servicio requerimos(orientado a conexión o no). El socket del proceso debe estar escuchando antes de conectarme, no funciona levantar a demanda.

La API de sockets agrega una nueva abstracción para las comunicaciones en red, el socket. Al igual que con los archivos, cada socket es identificado por un handle y la información relacionada es almacenada en la tabla de file descriptors.

Como es un stream va a parecer que leo y escribo de un archivo infinito

Sockets

- El servidor debe estar en ejecución ANTES de que el cliente intente conectarse
- El servidor debe tener un socket (como una «puerta») abierto, por el cual recibir y enviar mensajes (segmentos)
- Los clientes necesitan un socket
- Los clientes necesitan conocer IP y número de puerto del socket del servidor

Un mismo proceso puede escuchar en ambos protocolos al mismo tiempo para el mismo puerto, como DNS(TCP, UDP).

Creacion de un socket

```
int socket(int domain, int type, int protocol);
```

El dominio puede ser:

- AF_INET
- AF_INET6
- AF_UNIX
- AF_UNSPEC

El tipo puede usar:

- SOCK_DGRAM(UDP)
- SOCK_RAW(No agrega ningun encabezado, responsabilidad mia)
- SOCK_SEQPACKET
- SOCK_STREAM

El protocolo puede ser:

- IPPROTO_IP
- IPPROTO_IPV6
- IPPROTO_ICMP
- IPPROTO_RAW
- IPPROTO_TCP
- IPPROTO_UDP
- IPPROTO_SCTP

Aunque generalmente se deja en cero para que se seleccione el protocolo por defecto en base a domain y type.

Puede fallar por falta de sockets o simplemente por parametros invalidos.

En el ejemplo TCP, el **connect** se bloquea hasta que reciba el handshake, el servidor no se bloquea porque es mas importante que sea concurrente.

Si se quiere bloquear ciertas IP desde la aplicacion(sin firewall) hay que hacerlo despues del **accept**.

Socket I/O

La comunicacion en un socket es bidireccional. Se puede deshabilitar I/O con shutdown:

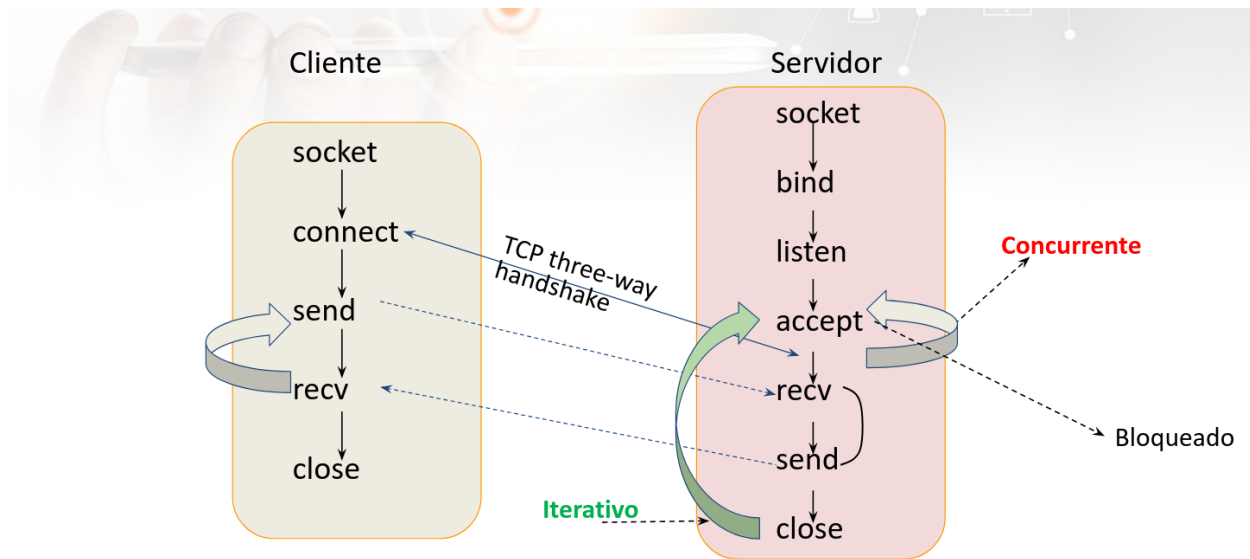


Figure 1: Ejemplo TCP

```
int shutdown(int sockfd, int how);
```

Posibles valores para how:

- SHUT_RD
- SHUT_WR
- SHUT_RDWR (El socket sigue, pero no lo voy a usar mas)

El SHUT significa que se manda un FIN.

Si quiero que el otro no envíe datos que no voy a poder leer pongo un window de 0

Formato de direcciones

Una dirección identifica un **socket endpoint**. El formato es específico para cada familia, y se castean a un formato genérico.

Resolución de nombres

Un cliente debe especificar la dirección de un servidor utilizando la estructura `sockaddr_in`. ¿Qué sucede si el programa cliente conoce sólo el FQDN del servidor?

- `inet_addr`: Convierte de notación con puntos ("200.132.2.15") a decimal.
- `gethostbyname`: Dado un string que contiene el FQDN de un host retorna una estructura que contiene –entre otras cosas– el IP del host en forma decimal.
- `getaddrinfo`: Más completa que la anterior.

`getaddrinfo` es la que vamos a usar pues permite especificar número de servicio y algunas cosas más.

Se puede usar tanto en el servidor como en el cliente.

Notar que el socket pasivo no lo cerramos nunca, va a estar vivo mientras lo este el servidor.

Diseno de servidores

Algoritmo para servidor iterativo no orientado a conexion

1. Crear un socket y ligarlo (bind) a un puerto
2. Leer un datagrama request de un cliente
3. Enviar datagrama/s como respuesta
4. Volver al punto 2

Un datagrama es un mensaje auto-suficiente

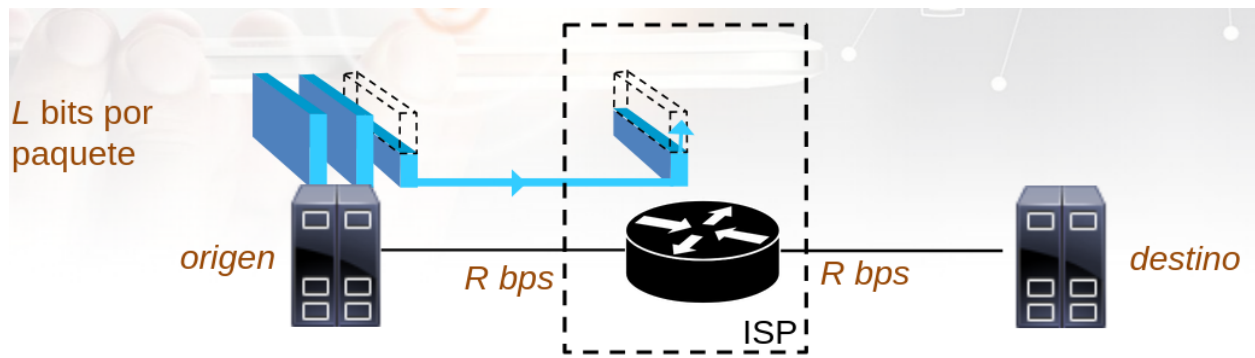
Algoritmo para servidor iterativo orientado a conexion

1. Crear un socket y ligarlo (bind) a un puerto
2. Aceptar un pedido de conexión a través del socket
3. Obtener un nuevo socket para la conexión
4. Leer un request del cliente
5. Enviar una respuesta
6. Si el cliente no finalizó, volver al punto 4
7. Cerrar el socket creado en punto 3
8. Volver al punto 2

Unidad 1: Introduccion

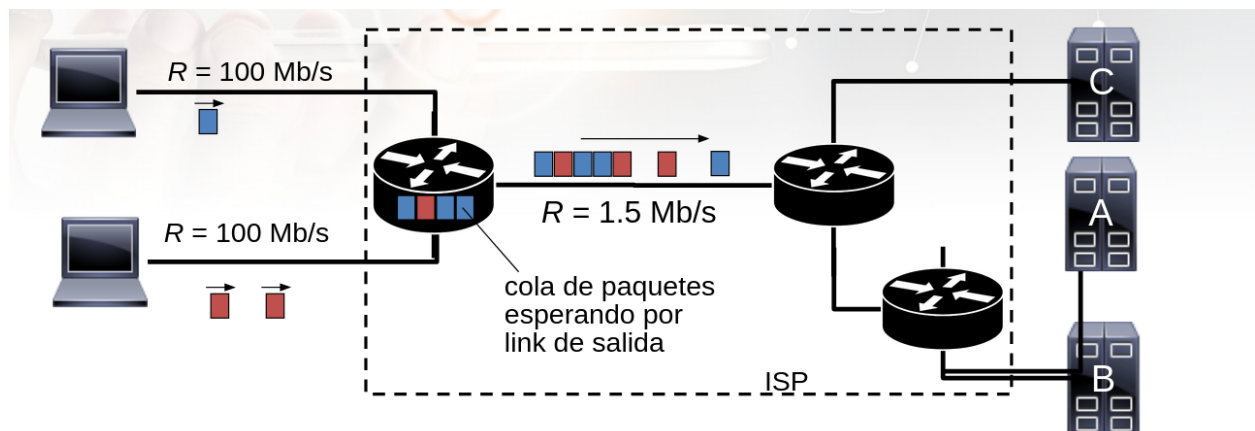
Las redes que vamos a ver es un conjunto interconectado de hosts, cada host es un equipo autonomo y no son necesariamente computadoras(pensar en IoT). El host envia datos en paquetes, de L bits de longitud.

Hay un problema, por ejemplo cuando contratamos un ISP, ya que nos venden la velocidad entre nuestra casa al ISP, pero no garantiza la misma velocidad a otros servicios. Ademas, si todos se conectan al mismo ISP, aparece un cuello de botella, haciendo que todo sea mas lento.



$$\text{Demora de transmisión de un paquete} = \text{tiempo necesario para transmitir paquete de } L \text{ bits} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

store and forward: el paquete debe arribar al router en forma completa antes de ser retransmitido



encolamiento y pérdida:

- ❖ Si la tasa del link de entrada es superior a la de transmisión en un determinado período:
 - se encolan paquetes, esperando por ser transmitidos
 - si el buffer está lleno, paquetes entrantes se "droppean"

Antes, tampoco había estándares, entonces sistemas de diferentes empresas podían no ser compatibles.

Modelo OSI - TCP/IP

Es un modelo de 7 capas, usando el protocolo TCP/IP, el cual se utilizó para estandarizar todo esto.

A la capa de red no le importa el motivo, solo le importa hacer llegar un paquete de una computadora a la otra.

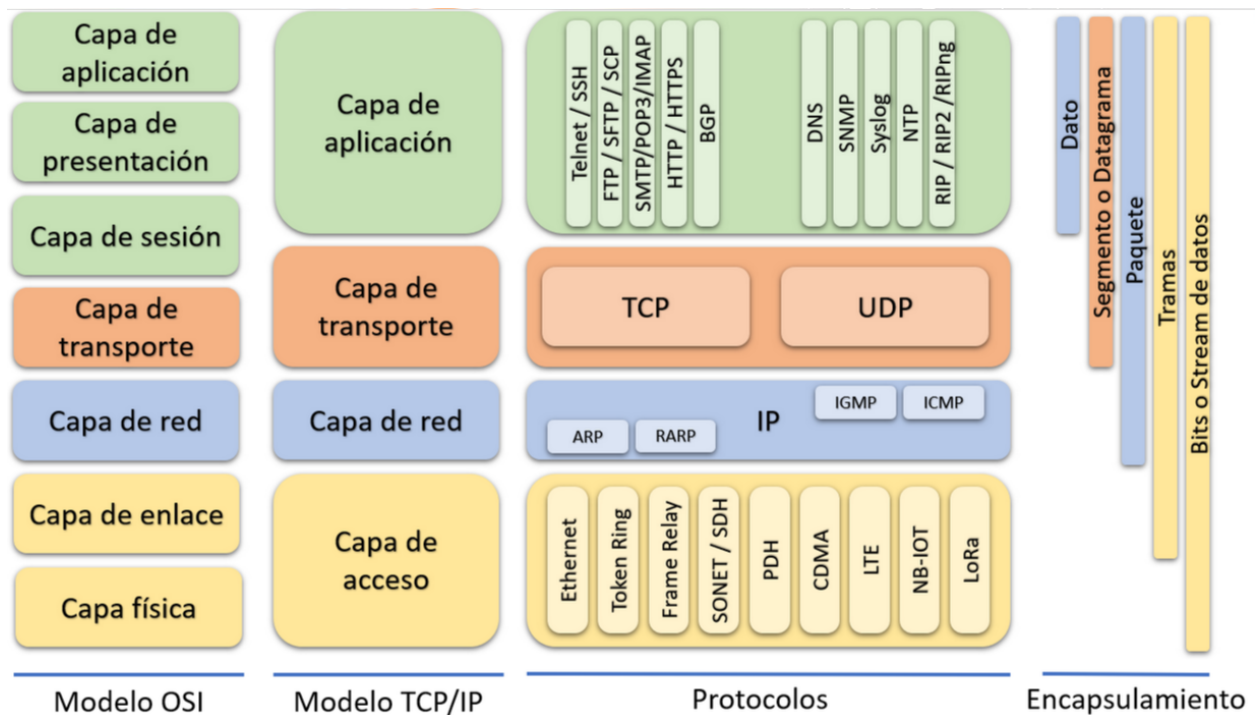


Figure 2: Modelo OSI - TCP/IP

Un protocolo es un conjunto de reglas que ambas puntas tienen que seguir

Estos protocolos pueden ser:

- **Confiables:** Confirma si la información fue recibida, utiliza acuse de recibo, reenvía información de ser necesario e informa al nivel superior si no se pudo enviar
- **No confiable:** No puede asegurar si el destinatario recibió o no la información enviable

Capa de enlaces

Tiene varios objetivos:

- Vincular hosts contiguos
- Controlar el acceso al medio físico
- Detectar y/o corregir errores básicos
- Encapsular los paquetes en tramas

Por ejemplo, una topología tipo **bus**. Para evitar colisiones, surgieron dos aparatos:

- **Hub:** Menos inteligente, trabaja en la capa física
- **Switch:** Más inteligente, trabaja en la capa de enlace. Asocia las MAC de las vocas(va aprendiendo).

Capa de ruteo

Hay que distribuir las IP alrededor del mundo, entonces, se dividió en zonas para que cada una realice la distribución. No se pueden conectar dos hosts que usan enlaces diferentes, para ello se necesita un router. El router de nuestras casas tiene 2 **interfaces**, pues hay una interfaz por red (WAN y LAN). El router

domiciliario es mucho mas que un simple router, pues internamente tiene un switch para el LAN que luego va a una de las interfaces. Sin embargo, a nivel capa de IP, todo esto es una misma red con la excepcion de WAN(red externa). Se podria decir que el switch tambien tiene 2 interfaces mas pues tiene 2.4GHz y 5.8GHz, entonces... tiene 6 interfaces el router? **No**, pues en un switch se llaman vocas(capa enlace), las interfaces son de la capa IP.

En conclusion, vamos a considerar que un router tiene dos interfacez, todo esto a nivel logico y de seguridad.

Capa de transporte: Puestos

El numero de puerto identifica a los procesos dentro de hosts.

Capa de aplicacion: soporte a aplicaciones

Aca aparecen los servicios mas conocidos: HTTP, POP3, SMTP, DNS, SSH, etc.

Comando que vamos a usar en todas las practicas: `tail -f /var/log/syslog` (-f es de follow)

Serializacion

- Se puede transmitir numeros entre computadoras? Si y no, dependera como codifica el numero(little endian o big endian) una computadora y como lo decodifica la otra. Para solucionar esto, en los protocolos de comunicacion se transmite todo en big endian. La mayoría de las computadoras hoy en dia usan little endian.
- Cuando tenemos que calcular la longitud de un string codificado en unicode, y tiene caracteres especiales, surgen problemas pues los caracteres podrian ocupar mas de un byte.

Unidad 2: HTTP

Antes, para obtener archivos se utilizaba un protocolo llamado **FTP**. En base a las referencias se debia acceder a otro sitio FTP y descargar el/los textos deseados. Surge la necesidad de aplicaciones basadas en **hipertextos**, para no tener que hacer todo manualmente, de ahi surge en HTTP.

HTTP 0.9(1991)

La primer version estaba basado en texto de recuperacion de informacion con un mecanismo request-response. Era contenido estatico y no mantiene el estado. Orientado a la **línea de comando**. Mas adelante se dieron cuenta que tenia potencial comercial, para que sirva para algo mas que papers cientificos.

TCP es de texto o binario? Es binario, mientras que HTTP es de texto.

HTTP 1.0(1996)

Ahora necesita ser **browser friendly**, entonces se agregan comandos como GET, POST, HEAD. Ademas, puede transmitir otras cosas que no sean hipertexto.

HTTP 1.1(1999)

Aparecen cabeceras en las peticiones, se agregaron los metodos PUT, DELETE, TRACE, OPTIONS. Negociacion de contenido, soporte de cache, conexiones persistentes. Ahora cada recurso se identifica con una **URI**(Uniform

Resource Identifier)

HTTP/2

Pasa a ser un protocolo binario, tiene compresion de headers, multiplexacion de recursos, server push, etc. Es mucho mas rapido que HTTP 1.1.

Comparacion: <http://www.http2demo.io/>

Porque alguien usaria HTTP 1.1? Algun cliente que no soporte HTTP/2 o porque no hay soporte para la tecnologia que tenemos que usar.

HTTP/3

Usa el protocolo **QUIC**(por ahi no lleguemos a verlo) como **transporte**, tiene menor latencia, multiplexacion mejorada, etc. QUIC es un protocolo de transporte pensado para usar on HTTP pues pide muchos recursos. Desde el punto de vista de la funcionalidad, es de transporte, pero se podria considerar de aplicacion.

HTTP/3 no usa TCP, usa QUIC

Recursos

Bloque de informacion identificado por su URI, puede ser un archivo(fisico) o generado por un programa(abstracto). La URI puede ser un URL o un URN. Con URN el archivo puede cambiar de lugar y el servidor se va a encargar de buscarlo, mientras que con URL se pierde.

Que diferencia hay entre una URL y una URI? URI es la clase abstracta para identificar recursos de forma univoca, mientras que URL y URN son clases concreta.

Los protocolos son publicos y se identifican con RFC(1630, 2396, 2718, 3305, 3986).

URI(RFC 3986 3.3)

`<scheme>://<authority><path>?<query>`

El path termina con el primer ? o # o si no hay mas caracteres. Puede ser relativo o absolutos. Si representa una aplicacion puede recibir parametros. El schema identifica el protocolo y la mayoría de los browser lo agrega por defecto. Pueden ser usadas para acceder recursos, ya sea por medio de URL o URN.

URL

`<scheme>://<user>:<password>@<host>:<port>/<path>?<query>`

Identifica un recurso por su ubicacion.

URN

Identifica un recurso por su nombre, no implica que el recurso exista o como acceder a el.

Por ejemplo:

`urn:isbn:0132856204`

Escribimos `www.clarin.com` y el browser se encarga de detectar que es HTTP. Luego el browser averigua la IP(dsp lo vemos) y se conecta mediante TCP al servidor.

En caso(por ejemplo) de AWS, no es factible asignarle una IP publica a cada cliente, lo que se hace es tener todas en el mismo servidor(misma IP publica) y se diferencias las paginas por la URI, entonces, a veces puede no funcionar solicitar una pagina web escribiendo directamente la IP.

Por otro lado, los headers en un request estan delimitados por un espacio(`\n\n`), esto aprovecha que es texto, sino no se podria hacer. Mientras que para saber cuando termina el cuerpo se indica en el header con `content-length`.

Esto funciona perfecto si es estatico, pero surgen problemas si es dinamico.

Request menos utilizados

- PUT
- TRACE
- OPTION
- DELETE

Respuestas

- 1XX: Informacion
- 2XX: Exito
- 3XX: Redireccion
- 4XX: Error en cliente
- 5XX: Error en servidor

La diferencia entre **moved permanently**(301) y **found**(302), a pesar de que ambos solo redireccionan, es que el primero es permanente, no va a cambiar. Por lo tanto, uno es una regla que no va a cambiar, permitiendo que el navegador se lo guarde y lo use para no gastarse en mandar de vuelta el request.

Tipos de informacion

HTTP solo transmite texto, utiliza **MIME** para describir contenido multimedia, cada objeto es etiquetado por el web server. Se define en el header `accept`.

- `text/html`
- `text/plain`
- `video/mp4`
- `image/jpeg`
- `application/pdf`
- etc.

GET vs POST

La idea es que el **GET** sea idempotente. Puede ser cacheado, el browser los mantiene en el historial, pueden ser **bookmarked**, tienen logitud acotada y solo son para recibir datos.

Idempotente es que puedo hacer **GET** todas las veces que pueda y el estado del servidor seguira siendo el mismo(**PUT** no es idempotente, puede cambiar el estado en el primer request y en el segundo el estado sera otro).

DELETE es idempotente tambien.

Mientras que el **POST** nunca es cacheado, no se mantienen en el historial del browser, no pueden ser bookmarked, no tienen longitud de datos maxima.

Headers HTTP

Como se menciona anteriormente, finaliza con una linea en blanco.

`<nombre>: <valor asociado>`

Tipos

- Generales
- De solicitudes
- De respuestas
- De contenido

Generalmente se envian en ese orden.

Headers generales

- **Cache-control:** Directivas para cache
- **Connection:** Se definen opciones de conexión
- **Date:** Fecha de creación del mensaje
- **Transfer-Encoding:** Indica el encoding de transferencia
- **Via:** Muestra la lista de intermediarios por los que pasó el mensaje.

Headers de solicitud

- **Accept:** Tipo de contenido aceptado por el cliente
- **Accept-Charset:** Charset (ISO-xxxx, UTF-8) aceptado por el cliente
- **Accept-Encoding:** Encoding (gzip, compress) aceptado por el cliente
- **Expect:** Comportamiento esperado del server frente al request.
- **From:** Email del usuario de la aplicación que generó el request.
- **Host:** Servidor y puerto destino del request.
- **If-Modified-Since:** Condiciona al request a la fecha indicada.
- **Referer:** URL del documento que generó el request
- **User-Agent:** Aplicación que generó el request
- **Upgrade:** solicita que use otro protocolo
- **Range:** solicita un rango (en bytes) del recurso

Referer esta mal escrito(va con *rr*), fue un error y quedo asi :/.

accept es lo que le mandamos al servidor, pero despues el servidor especifica cual te manda en **content-type**.

Headers de respuesta

- **Age:** Estimación en segundos del tiempo que fue generada la respuesta en el server.
- **Connection:** close, keep-alive
- **Location:** URI a redireccionar
- **Retry-After:** Tiempo de delay para reintento
- **Server:** Descripción del software del server.
- **Authorization:** indica que el recurso necesita autorización

Headers de contenido

- **Allow:** Métodos aplicables al recurso.
- **Content-Encoding**
- **Content-Length**
- **Content-Location**
- **Content-MD5**
- **Content-Type**
- **Expires:** Fecha de expiración del recurso.
- **Last-Modified:** Fecha de modificación del recurso

Proxy servers

Forward proxy, es algo que esta en el medio, pueden ser para molestar tambien. Actua como intermediario entre una aplicacion cliente y un web server. Puede ser:

- **Explicito:** Lo configuramos nosotros, sabemos de su existencia.
- **Transparente:** A traves de un firewall, no nos damos cuenta que esta.

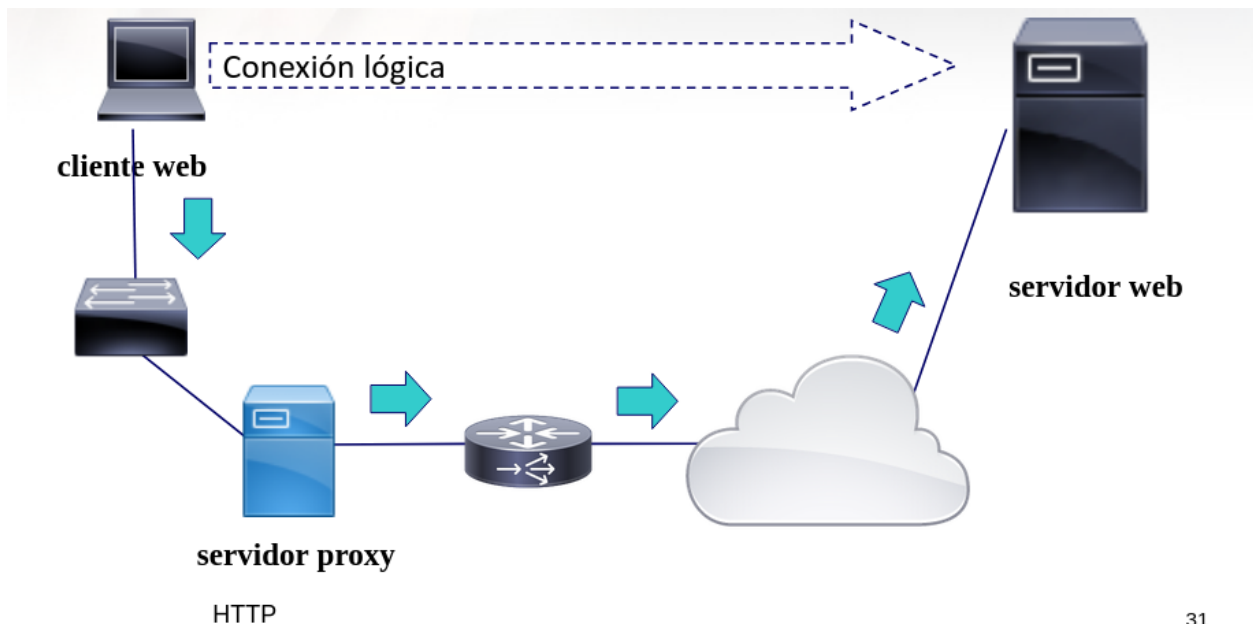


Figure 3: Proxy server

Tiene varias finalidades:

- Controlar el acceso a determinados sitios o paginas
- Almacenar en cache recientes consultas, haciendo que sean mas rapida pues usamos la red local

Sin embargo, el cache puede causar contenido desactualizado, por lo que el que genera el recurso decide cuanto tiempo dura el cache. Setea el parametro **max-age**.

Proxy reverso

El proxy directo esta del lado del cliente, mientras que el reverso, **reverse proxy** esta del lado del servidor. Redirecciona el trafico internamente a distintos servidores, por ejemplo, **NGINX**.

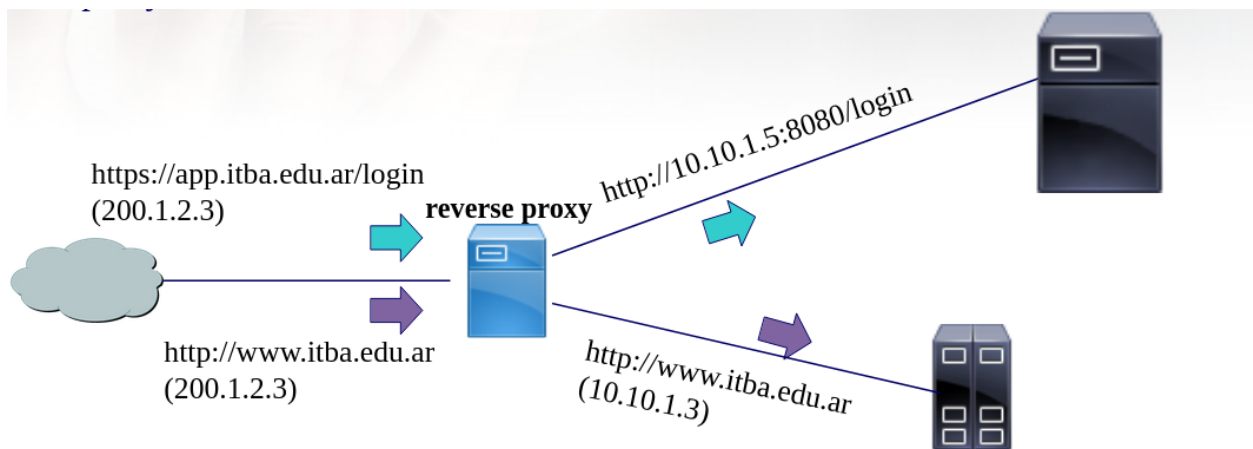


Figure 4: Reverse proxy server

En el cache del browser/proxy esta **last-modified**, **max-age**, etc.

Cookies

La idea es que cuando pida un recurso, en el header esten las cookies. Entonces, cuando el servidor me manda cookies (**set-cookie** res header), a ese servidor en particular, cuando pida un recurso le voy a mandar las cookies(**cookie** req header).

Es un texto de informacion enviado por el servidor y almacenado en el servidor, se usa para mantener un estado entre el cliente y el servidor. Tiene dos tipos de persistencia:

- Session cookie
- Persistent cookie

Tambien hay **third-party** cookie y **secure** cookie.

Un RFC es un pedido a un estandar, se publica, es revisado por un monton de gente. Si se aprueba pasa a ser un estandar.

Curl

A la hora de descargar archivos:

```
curl <url-archivo> --output <archivo>
```

Si lo detengo a la mitad y vuelvo a correr el comando, curl va a empezar de nuevo. Para que continúe con lo que ya tenía hay que agregar `--continue-at`:

```
curl <url-archivo> --output <archivo-nuevo> --continue-at <archivo>
```

Curl asume que el archivo que le das para continuar es la primer parte de la descarga, caso contrario se unen dos archivos y queda algo raro que no sirve.

Es muy buena la documentación de **Mozilla** para referenciar.

Unidad 3: Resolución de nombres

Lo que hace HTTP, primero se crea una conexión mediante TCP, luego se pueden mandar los request. Esta conexión se hace a partir de la IP, entonces como se obtienen las IP a partir del domain? Hay dos formas, una usa DNS.

Resolución de nombres - DNS

Proceso de traducir un nombre de dominio a una dirección IP.

Un **dominio** es una colección de computadores que pueden ser accedidas usando un nombre en común. Un **nombre de dominio** hace referencia al nombre de múltiples hosts que son referenciados colectivamente

Las computadoras de un mismo dominio no tienen que estar en el mismo lugar. No tienen nada que ver.

No todos pueden encargarse de la resolución de nombre, pero si se pueden tener subdominios. Por ejemplo, solo una organización da los .com, pero después yo puedo tener mis propios subdominios como subdominio.ejemplo.com.

No siempre se tiene que acceder a un DNS, podrían estar localmente en `/etc/hosts`, si se encuentra, se usa eso. Ahí se encuentra `localhost`, pues no conviene que tenga una regla especial, es mejor que se comporte como todos los demás. También sirve si queremos bloquear alguna página:

```
127.0.0.1 chatgpt.com
```

En examen pueden haber preguntas de qué códigos se devuelve en cierta situación, por ejemplo, si mapeo mi página web a clarin.com en mi computadora, que pasa cuando el servidor ve que el host del request no es el suyo? Devuelve algún 4XX.

En caso de FQDN, se obtiene su número de IP del DNS o de `/etc/resolv.conf`. Se pueden especificar:

- `nameserver <direcciones>`
- `domain <nombre>`
- `search <dominios>`
- `sortlist <red\[mascara\]>`
- `option <opcion>`

Los `nameserver` tienen que ya estar especificados por su IP, son los servidores DNS. Cuanto duran las resoluciones en la cache? Depende del **timeout** que te devuelva el servidor DNS junto a la IP.

Linux tiene las utilidades `dig` y `host` para mostrar como la computadora, a partir de un nombre de dominio, encuentra la IP (reverse DNS) o al revés.

Si un día alguna página web cambia de IP (cambio de proveedor de internet, cambio de ubicación, etc). Entonces, si me quiero conectar, mi computadora va a intentar de acceder a la vieja IP, algo similar va a pasar en el cache de todos los DNS. Entonces, va a haber que notificar a **a.riu.edu.ar** pues es el root server que maneja **edu.ar**, no hace falta notificar al de **ar**.

Como se consiguen los nombres

Es un sistema jerárquico y distribución para la resolución de nombres de dominio en direcciones IP (y viceversa). Se lee al revés, empieza la jerarquía más alta por la derecha. Es **jerárquico y distribuido** porque delega a los de abajo.

El nombre **completo** en realidad es un **.**, es el top-level domain.

Entonces, hay distintos tipos de servidores DNS:

- **Recursivos:** Realizan la consulta completa en nombre del cliente
- **Autorizados:** Contiene la información oficial
- **Raíz DNS:** Dominio padre

La distribución puede ser vista como un árbol invertido donde la raíz hace referencia a servidores de nombre raíz (root name servers).

Los **root name servers** se usan cuando el servidor de nombres local no pueden resolver un nombre. El root name server luego:

1. Contacta a un name server autorizado si no conoce la respuesta
2. Almacena
3. Retorna la respuesta al servidor de nombres local

Hay 13 de estos en total.

Generalmente el **DNS server local** es el router.

Las preguntas y las respuestas de los servidores DNS son UDP.

Cache

Una vez que el servidor de nombres aprende el mapeo, lo almacena en cache.

—

Los servidores raíz DNS son operados por IETF. Cuando alguien obtiene un nombre de dominio es el responsable de mantener, en un servidor DNS primary (antes master), actualizado ese dominio (Completa la idea de lo que ocurre cuando cambio mi IP).

En Unix y Linux DNS es implementado generalmente con **BIND** (Berkeley Internet Name Domain).

Configuración de servidor DNS

Hay cuatro niveles de configuración:

- Resolver only systems
- Caching-Only
- Primary (Antes master)

- Secondary(Antes slave)

Registros DNS

DNS es una base de datos distribuida de **resource records**(RR). Los registros son:

- A
- NS
- CNAME
- MX
- AAAA
- SOA
- SRV
- RP
- TXT
- SPF

Los registros asocian el request a distintas cosas, por ejemplo, A devuelve la IP.

SRV es tipo MX, pero mas generico

Formato RR

(name, value, tyype, ttl)

El @ surge para separar el nombre del dominio del nombre del usuario, sino se tornaba confuso.

El tiempo de vida minimo para respuestas negativas se refiere a cuando el servidor tiene que ir a preguntar a otros por un nombre de dominio. Si no lo encuentra, durante las proximas, por ejemplo, 3 horas no volvera intentar a preguntar por el nombre a otros(va a devolver que no lo encontro).

Split-horizon

Un nombre se resuelve con distintas IP dependiendo del origen de la consulta. Por ejemplo, desde el ITBA el servidor de pampero es 10.16.1.103, mientras que desde afuera es 200.5.121.139.

No es si la pregunta se hace desde adentro o desde afuera, sino que donde esta el servidor DNS, si esta afuera me va a dar la IP de afuera por mas que yo este adentro del ITBA.

DNS dinamico

DDNS permite actualizar en forma dinamica un servidor DNS. Introducido en BIND a partir de la version 9. Si cambia la IP, el servidor se va a actualizar automaticamente.

DNS spoofing

Es el arte de lograr que un registro DNS apunte a un IP que no sea el que deberia apuntar. De manera que algun actor malicioso podria crear una copia de la pagina del ITBA y hacer DNS spoofing para que el nombre de dominio del ITBA apunte a su pagina. Esta seria una manera de realizar un ataque de phishing. Por eso es importante preguntar para corroborar la resolucion DNS.

Es comun que muchas empresas sigan usando versiones viejas de software como BIND.

Metodo: Enviar respuesta sin recibir pregunta

Se envia como falsa respuesta el IP de algun nombre de dominio. Se debe verificar que la respuesta corresponda con alguna pregunta. DNS establece que los mensajes de pregunta y respuesta tengan un mismo ID que los identifique.

Metodo: DNS sniffer

Person in the middle(no man in the middle), se obtiene el ID para poder responder con una IP falsa en nombre del servidor DNS.

Metodo: DNS cache poisoning

Inyecta otras resoluciones a una simple pregunta para ensuciar el cache del servidor. Pisa las resoluciones anteriores.

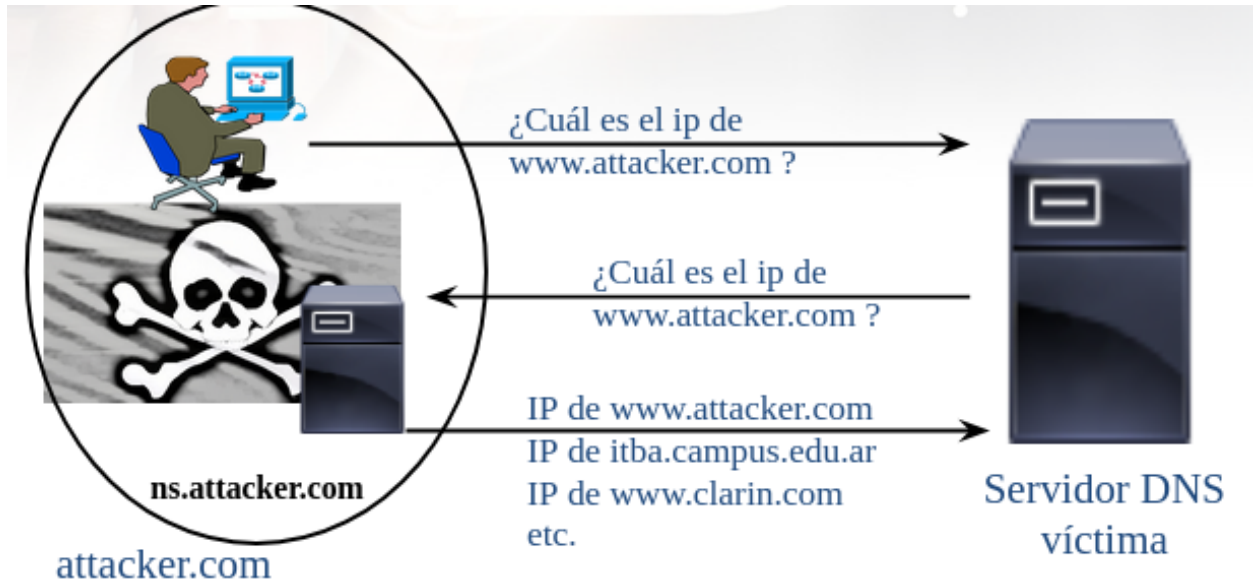


Figure 5: DNS Poisoning

Si pedis el servidor MX por lo general tambien te devuelve A y AAAA pues es muy probable que sino harias otra consulta para averiguar eso.

Notas practica

- Como se sabe si un servidor es root? Basicamente, hay 13 IPs hardcodeadas.
- Los autoritativos nunca hablan con los clientes, ese es el trabajo de los cache, estos si hablan con los autoritativos.
- Si preguntan porque no funciona una zona(porque a x no le aparece?), generalmente se debe a un error boludo, tipo poner mal el serial.

Unidad 4: Correo electronico

Cuando empezo no se considero que iba a ser usado por personas individuales, entonces no habia ningun tipo de seguridad.

Antes, en los ambientes Unix un usuario podia mandarle un mensaje a otro de forma asincronica. Cada usuario tenia una carpeta donde se le dejaban los mensajes , es decir, estaba pensado a los usuarios de un mismo sistema.

Luego, para permitir mandar mails a personas que esten en otros dominios, se creo un protocolo. Asi surgio el Simple Mail Transfer Protocol(SMTP) en 1982. Se encarga de dejar en la carpeta de mail de cada usuario el mail enviado a dicho domino.

Se usa el registro MX de los servidores DNS para resolver los request de mails. SMTP usa TCP, la tranferencia es directa y los mensajes son en US-ASCII. Tiene tres fases:

- Handshaking
- Transferencia de mensajes
- Cierre

Envio de datos binarios

Si no soporta otra cosa que sea US-ASCII(de 7 bits), como se envian binarios por ejemplo? Se codifica antes para que sea un mail valido, luego se decodifica. Se debe codificar como si fuera texto con \n en cada linea u otros.

Simil a HTTP, devuelve codigos para indicar el estado de la transferencia, por ejemplo, 250 es que todo estuvo OK.

Es un protocolo de texto. Se puede usar netcat para crear una conexion y ver esto.

Aca si da lo mismo si uso la IP o el host, a diferencia de HTTP.

Como mandar mail con netcar

Primero vemos cual es el servidor de mail del dominio al que queremos contactar:

```
dig MX <dominio>
```

Luego, creamos una conexion con netcar:

```
netcat <Servidor MX> 25
```

Una vez tengamos una conexion asegurada:

```
EHLO
MAIL FROM <mail>
RCPT TO <mail>
DATA
SUBJECT <subject>
FROM <mail>
\n
<Contenido del mail>
```

EXIT

Importante terminar con un .. Es interesante que puedo poner cualquier mail y puedo usar el mail de otro como el que envia. Sin embargo, estos mails suelen terminar en spam pues les falta un monton de informacion.

Si lo pongo al mail del itba me lo va a rechazar porque el servidor de google(que es el que usa el itba) hace una verificacion extra, esta es validar la IP de origen.

Las cadenas de mails se logran mediante un id que tiene cada mail.

Los MX reciben mail, pero hay otros registros que guardan los servidores de mails autorizados. Esto es lo que hacen la mayoría de los servidores para verificar los mails.

Si es algo que no se puede enviar(TO invalido), el campo FROM va vacio para evitar un ciclo infinito, que lo que ocurriria si este es invalido.

Protocolos de entrega final al usuario

- POP3
- IMAP

IMAP evita perder todos los mails pues se guardan en el servidor siempre, caso contrario si yo pierdo o se rompe mi disco duro, voy a perder todos los mails. Hoy en dia practicamente no se usa IMAP, es porque hoy en dia generalmente se usa una aplicacion web para ver los mails(y los mails estan en el servidor). En todo caso la aplicacion web usaria IMAP. Esta practica se conoce como webmail.

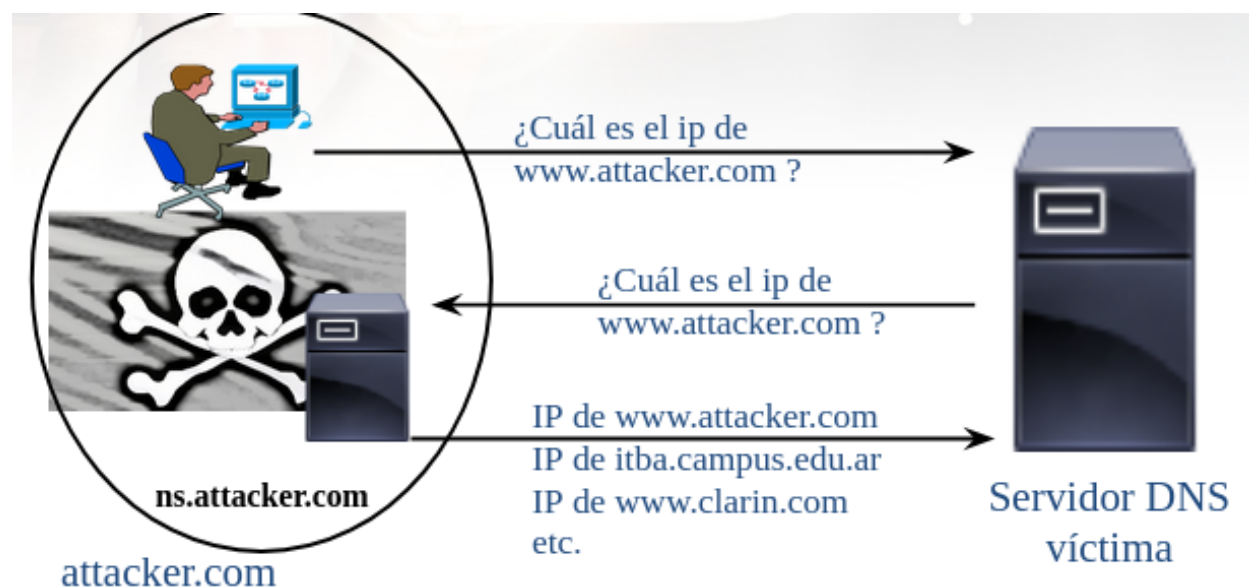


Figure 6: Webmail

Encoding: Base64

Es un metodo de codificacion simple:

- Cada grupo de 3 bytes es codificado como 4 bytes, cada uno conteniendo solo 6 bits de datos.
- Estos son enviados como ASCII de 7 bits.

SPAM

Surge de un sketch de Monty Python.

Como evitarlo:

- Allow list y deny list
- Grey listing(porque antes eran white y black list)

Se clasifican con **filtros bayesianos**. Gray list es que es dudoso, no le acepta ni rechaza la conexion, sino que dice que intente mas tarde. Entonces, si es de SPAM y sigue volviendo a mandar mails entonces para a la lista negra. Una persona haria lo logico de volver a intentar mas tarde.

Tecnicas para autenticar mails

- SPF(Sender Policy Framework)
- DKIM(DomainKey Identified Mail)

SPF

El servidor DNS de un dominio tiene registros para indicar cuales son los servidores autorizados para mandar mails de ese dominio.

DKIM

El servidor que envia el mail verifica que el usuario realmente es el de su dominio, se le agrega un sello para garantizar que no fue alterado.

Notas practica

- Soluciona la necesidad de que ambos host deban estar online para recibir o enviar mensajes, se lo delega a los servidores de mail.
- TXT indica que servidores estan autorizados a enviar mails a nuestro servidor. Importa el orden en los comandos, se puede especificar un rango de IPs. **-all** indica que ignore todo lo demas.
- El primero en hablar no es el cliente sino que el servidor.
- **HELO** es con la version vieja. **EHLO** es con la version nueva que soporta encabezados.

Unidad 5: TLS/SSL

SSL

Se quiere encriptar datos desde y hacia el servidor, un Web Server en este caso usa SSL. HTTP no ofrecia nada de seguridad o identificacion.

Se conoce como **Secure Sockets Layer**, va antes de la capa de transporte, es la capa que se encargara de encriptacion y seguridad. Se mete en el medio de un protocolo no seguro y el transporte. Es agnostico al protocolo que este encriptando, **HTTPS** no es un protocolo en si, es simplemente HTTP con esta capa extra.

Claves asimetricas

No hay una unica clave que encripta y desencripta. Sino que hay una publica y una privada, un mensaje codificado con la clave publica solo se puede desencriptar con la clave privada.

Entonces es facil, si quiero comunicar a y b, es cuestion de que a tenga la clave publica de b y viceversa.

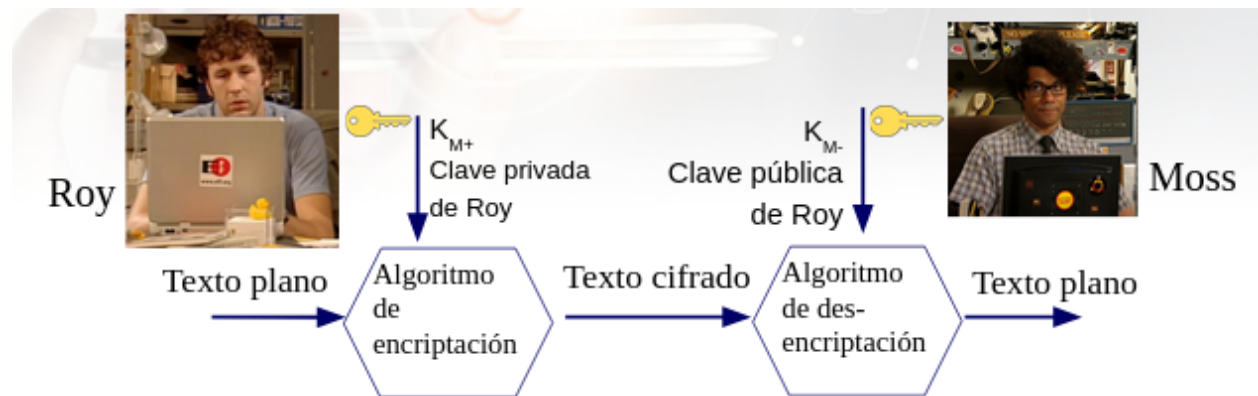


Figure 7: SSL

La primera vez debe ser la persona actual a la que queremos enviar informacion encriptada, pues se transfiere en HTTP simple. Como nos aseguramos que el que este del otro lado sea la persona que creemos que es? Se encripta una passphrase con la clave publica, si el otro responde con la misma clave encriptada con mi clave publica. Si son iguales, entonces esta bien.

En el browser, al usar HTTPS, en vez de preguntarnos cada vez si confiamos en el servidor que esta del otro lado, se usan certificados emitidos por fuentes confiables que el browser usa para chequear si es valida la clave publica.

Se llaman **entidades certificantes**

En HTTPS se utiliza SSL solamente en la parte inicial para transmitir una clave simetrica y usar esa durante la conexion. Esto se debe a que SSL es bastante pesado.

Se crea una clave simetrica por cada sesion

Tipos de SSL handshake

- Full handshake
- Session resumption

Session resumption acelera el proceso

TLS

Transport Layer Security es lo mismo que SSL, pero inicia la conexión segura de otra manera. Comienza con un **hello** inseguro y luego cambia a una conexión segura. No tiene la necesidad de tener un puerto específico, se crea la conexión segura en el mismo puerto, no hace falta.

En SSL hay un comando `redirect` para pasar a otro puerto donde se realizara la conexión segura.

DNS over TLS(DoT)

Usa TCP y se usa para crear una conexión segura con DNS, esto evita tener una persona en el medio y ataques similares.

DNS over HTTPS(DoH)

Es otra manera de tener una conexión mas segura a un servidor DNS, es menos eficiente.

HTTPs y proxy HTTP

Los proxies HTTP se utilizan para cachar contenido, filtrar acceso y mejorar rendimiento. Cuando el tráfico es HTTP, el proxy puede inspeccionar, modificar y almacenar las respuestas.

Con HTTPS el tráfico esta cifrado entre el cliente y el servidor, lo que impide que un proxy vea el contenido directamente. Entonces, como logran los proxies bloquear sitios o cachear contenido cifrado? Con MITM.

Intercepcion HTTPS con MITM

Se conoce como **Man-in-the-middle**, es la tecnica que usan los proxies para inspeccionar HTTPS. Se coloca entre vos y el servidor, el proxy tendra una conexión HTTPS con vos y otra con el servidor, pasando los paquetes entre los dos y al mismo tiempo tener acceso a eso.

Deteccion

- Verificar el certificado SSL en el navegador.
- Si el emisor no es una entidad oficial podria ser un MITM
- Se pueden usar herramientas como wireshark para detectarlo

Limitaciones

- No funciona si el cliente no confia en el certificado del proxy
- Sitios con HSTS(HTTP Strict Transport Security) pueden rechazar la conexión con certificados no confiables
- Algunas aplicaciones y navegadores alertan sobre certificados sospechosos

Let's encrypt usa una query DNS para saber que dominios son validos, los dominios los saca de nuestra configuracion de nginx.

Unidad 6: Protocolos de transporte

DNS puede usar TCP y UDP.

Proveen la comunicacion logica entre dos procesos que corren en distintos hosts. Se ejecutan en las **puntas finales**, hay varios:

- UDP
- TCP
- SCTP(No se usa tanto)

Ninguno, UDP y TCP, ofrecen ni un minimo de demora o latencia, ni minimo de ancho de banda.

Se implementan con sockets, cada proceso tiene un socket. Estan pensados como file descriptors, entonces para hablar con otro host se usa un socket. Es mas, usan la misma tabla que los file descriptors(pues usa los mismos system calls).

Multiplexacion

(en la presentacion de campus hay mas data)

Gran parte de los puertos estan definidos tanto para TCP como para UDP, aunque algunas aplicaciones utilicen solo uno de ellos.

Si no somos root no podemos abrir puertos menores a 255(reservados para aplicaciones publicas).

Demultiplexacion sin conexion

Se usa para saber para que proceso/socket es lo que llego.

- Crear socket con numero de puerto.
- El socket se identifica por el par <IP destino, Puerto destino>
- Cuando el host recibe un segmento UDP -> Dirige el segmento al socket que atiende
- Datagramas con distinto IP origen son atendidos por el mismo socket

UDP

UDP transporta datos de manera no confiable entre hosts.

- No orientado a conexión
- No confiable ##- No ofrece verificación de software para la entrega de segmentos
- No reensambla los mensajes entrantes
- No utiliza acuses de recibo (ACK)
- No proporciona control de flujo

Demultiplexacion orientado a conexion

Seria TCP. Cada socket identificado por:

- IP origen
- Puerto origen
- IP destino
- Puerto destino

Host que recibe usa los 4 valores para dirigir el segmento al proceso que corresponde Conexiones simultáneas: c/u con su socket

TCP

Si una maquina tiene dos procesos intentando de conectarse al mismo servidor lo que ocurre es que cambia el puerto en el servidor, manteniendo el puerto de origen.

Repaso conexion confiable

Si el protocolo de red es confiable

- No se alteran bits
- No se pierden paquetes
- Los paquetes llegan en orden

Entonces, la lógica a implementar es muy simple

IP no es confiable.

Si hay errores se complica, pues hay que avisar que llego y que no llego, entonces los algoritmos para **Sender** y **Receiver** no son tan simples. Pero se puede recuperar con **ACK** y con **NAK**, pero se pueden perder esos tambien! Entonces, se manda otro **ACK**, pero el otro podria si haberlo recibido, es por eso que todo paquete debe estar identificado.

Hay algunos controles que se pueden implementar:

- **Stop & wait:** Es trivial, manda **ACK** y se bloquea, pone una alarma en caso de que no llegue y vuelve a mandar **ACK**. Es facil de programar.
- **Pipelining:** Se envian los paquetes y al final se manda un **ACK** diciendo hasta donde recibio. Si se recibio todo termino. Un poco mas complicado, pero mucho mas eficiente. Cada envio tiene una alarma de un tiempo, si no llega el **ACK** hay un error.

TCP no los numera 1, 2, 3... sino que usa bytes, es a nivel de desplazamiento de bytes que esta enviando. El primero no empieza en 0. Se coloca en el campo **Sequence number** del header.

El flag **URG** indica que si hay algun paquete urgente, que los mande primero. Esto no se usa mucho. El flag **RST** corta la conexion de una manera abrupta. En cambio, **FIN** indica que una punta no tiene nada mas que decir, pero se queda escuchando por si la otra punta todavia tiene algo que decir(mas largo que **RST**). Por otro lado, **CRC** es como un checksum, paridad.

TCP ofrece un circuito virtual entre aplicaciones de usuario final.

- Orientado a conexión
- Confiable
- Divide los mensajes salientes en segmentos
- Reensambla los mensajes en el host destino
- Vuelve a enviar lo que no se ha recibido
- Control de flujo (rfc 7323)
- Control de congestión (rfc 5681)

El cliente es el que inicia la conexion, es la unica distincion que usa TCP.

En el tercer **ACK** el cliente ya puede enviar datos.

Si esta la conexión y no tengo nada para mandar, de ambos lados se mandan datos **keep-alive**. Son paquetes ACK vacíos para mantener la conexión.

Si se manda el ACK 120 y se envía información **seq=180**, este se guarda en el buffer. Luego, se puede recibir el **seq=120** esperado. Si se vuelve a mandar el 180, se ignora.

El FIN lo puede mandar cualquiera de los dos.

Control de flujo

Si uno manda paquetes y se llena el buffer porque el otro está ocupado con otra cosa, no puede pisar el buffer porque ya mandó el ACK. Tampoco se pueden tirar los paquetes porque estaría mal. Lo que se usa es el size del buffer que se envió en uno de los paquetes (en el campo **window**), entonces no se envían más datos que puedan entrar en el buffer del que va a recibirlos.

El problema es que **window** solo puede especificar hasta un buffer de 64KB, lo cual es inaceptable hoy en día con las velocidades de internet actuales. Lo que se usa son los **options** para negociar y especificar que **window** indica el size con un multiplicador. Es una de las aplicaciones de **options**.

Gracias a esto se evitó armar un TCP nuevo y migrar todo.

Control de congestión

Puede ocurrir que haya un cuello de botella en algún punto de la red, entonces, se pierden paquetes. En teoría, la capa de arriba no debería encargarse de la capa de abajo (en este caso, la capa de red). Sin embargo, se puede tener información de la capa inferior. TCP se puede dar cuenta cuando hay congestión, como hoy en día es poco probable que se pierda un paquete por errores de red, la causa más probable es que hay congestión (y por eso llega el timeout). Entonces, una vez detectado eso, se mandan los paquetes más lento, como todos hacen esto se libera la congestión. Esto se conoce como **AIMD** (Additive Increase Multiplicative Decrease).

La idea es que nadie se haga el vivo, suele ser el caso, pero si ocurre hay forma de detectar si se siguen mandando paquetes a alta velocidad.

Como se decide un valor de **timeout** razonable? Se usa cuando tardó en ir y volver el handshake, de ahí tengo una cota inferior de cuando tardara, uso eso como **timeout**.

Como decidir el **timeout** no aparece en el RFC pues depende de la implementación.

NAPT (aka NAT)

SNAT

Hasta mediados de los 90s para usar internet cada host necesitaba tener una IP pública. Pero rápidamente IPv4 se quedó sin IPs y IPv6 tardaría décadas en terminar de implementarse globalmente.

Por lo tanto, surgió NAT, de manera que si en una red local se hace un pedido al internet, el router va a registrar la IP local y un puerto. Entonces, cuando sale el pedido al internet es con la IP pública y el puerto asignado.

Sale de la red interna a la externa.

DNAT

Port forwarding, configura NAT para cambiar el puerto de destino. Esto permite implementar balance de carga, teniendo n servidores con una unica IP publica.

Viene de la red externa a la interna.

Firewall

Un firewall opera en el nivel mas bajo de red:

- Evita acceso no autorizado a la Intranet
- Permite definir que trafico entra y sale de nuestra red

Lo vamos a ver mas adelante

Notas practica

- UDP es un wrapper de IP.
- UDP tiene algo muy importante, que usa puertos(TCP tambien). Esto permite identificar al proceso.
- Hay que saber configurar la maquina virtual, mas especificamente la red:
 - **NAT**: Permite vincular la red interna del VM con el internet, basicamente pone un router que conecta con el adaptador de red de la maquina real. Se pueden tener varias VMs conectadas a la misma red interna.
 - **Host-only**: Parecido a NAT, pero a la maquina host tambien le pone un adaptador virtual entonces, la VM puede hablar con el host.
 - **Bridged adapter**: La maquina deja de estar en su red interna y pasa a ser parte de la red real. Usa la enp1s0. En la maquina virtual siempre se vera como wired.
- xinetd, muy util.
- X11 es un protocolo que usa unix sockets.
- Quien manda el primer byte una vez se establecio la conexion(sin contar el handshake)? El servidor
- La window no es solo cuanto acepta el otro lado, sino que es cuanto puedo mandar antes de parar, cuanto esta en cable.
- En 1987 hubo algunos accidentes en internet, cada tanto el internet colapsaba. Se arreglaba reiniciando todos los routes. Entonces, observaban que la velocidad bajaba un monton antes de caerse. Lo que ocurrio fue que los routers encolaban los paquetes para no perderlos, entonces, se llenaban las colas y se perdian paquetes. Basicamente, no llegaban, se mandaban de nuevo, se multiplicaba la cantidad de informacion y colapsaba internet. Asi surge el control de congestion, sin esto el internet no funcionaria. Simplemente se empieza con un window size chico y se sube hasta que se pierdan paquetes.

Telnet

Predecesor a ssh, deprecado, va todo en texto plano, hasta la password. Para instalarlo:

```
sudo apt install telnetd
```

SCTP

Disenado originalmente para transportar telefonía sobre IP:

- Confiabilidad
- Control de flujo
- Secuenciación
- Orientado a mensaje
- Permite aceptar mensajes fuera de orden
- **Multihoming**, origen y/o destino con mas de una IP
- Permite definir flujos paralelos, solucionar **Head Of the Line blocking**

Cualquier cosa que sea una pregunta sencilla conviene usar UDP.

Actualmente, se usa TCP incluso para cosas sencillas por motivos de seguridad, la baja de performance es aceptable.

SCTP es mas moderno que UDP y TCP(pero mas complejo), soluciona el problema de TCP de perder conexión si se corta la conexión o me cambio de red. Venia a solucionar todos los problemas de TCP y UDP. SCTP permite tener dos conexiones simultaneas, por ejemplo, un socket por wifi y otro por ethernet(se puede elegir una o la otra tambien).

Head Of Line Blocking significa que implementa una conexión por recurso, de manera que pueden ser pedidos en distinto momento y uno no bloquea a los demas por tener que ir en orden. Es orientado a mensaje pues cada mensaje tiene un buffer propio, no hace falta quedarse esperando a que el mensaje que estoy leyendo llegue completo.

Inicio protegido

Tiene un 4-way handshake, mucho mas seguro y a diferencia de TCP, no reserva recursos para la conexión hasta que recibe un COOKIE-ECHO. De todas formas, con SCTP se puede empezar a recibir información a partir del primer mensaje.

Fin

A diferencia de TCP, si uno termina, se termina la conexión.

Porque no se usa?

- Compatibilidad limitada
- Restricciones en firewalls y NATs
- Solo se beneficia en comparación con TCP, UDP no tanto
- Falta de soporte en navegadores y aplicaciones
- HTTP/2
- HTTP/3

QUIC usa UDP y es utilizado por HTTP/3, es un protocolo de aplicación que funciona como transporte, lo cual hace algo similar a SCTP y es mas practico.

Unidad 7: Capa de red

Los protocolos de red corren en varios host: PC, routers, servidores, etc. El objetivo de la capa de red es trasladar paquetes de un host a otro, que pueden estar en distintas redes. Los routers manejan una tabla de ruteo para enviar los paquetes(en la capa de red) de un lado a otro. La capa de red podria ofrecer:

- Entrega garantizada
- Demora minima garantizada
- Orientada a conexion
- Entrega en orden
- etc.

Hubo muchas tecnologias que ofrecian todo eso, surgio de cuando se utilizaban las redes telefonicas el concepto de **Virtual Circuits Networks**. Era orientado a la conexion y enviaba todo en orden, pero era mas complejo y encima el primero en llegar se copaba todo el ancho de banda hasta que terminara. Entonces, tampoco servia que los protocolos de transporte se ocuparan de eso, de manera que era mas simple dejar que la capa de transporte se encargue de eso.

Internet es simplemente un tipo de red, mas especificamente **Datagram Network**, un paquete puede tomar un camino y el proximo otro sin problema, pueden llegar desordenados y luego los protocolos de transportes tienen que ocuparse de ordenarlos.

Direcciones en tabla de ruteo

No hace falta escanear toda la IP, basta con leer la primer parte hasta que sepamos a donde tiene que ir, el prefijo es lo que se conoce como mascara de red. Manda el paquete a la interfaz que mas matchee la direccion.



Figure 8: Subnets

Hay distintos tipos de redes:

- **Loopback:** 127...
- **Privadas:**
 - 10..,
 - 172.16.0.0 a 172.31.255.255

– 192.168...

- **Link local:** 170.254.0.0 a 169.254.255.255

Ejemplo tabla de red:

Destination	Mask	Gateway	Interface
20.0.0.0	255.255.255.0	20.0.0.1	eth0

En el super, el carrito es el enlace, la interfaz seria la caja para que el paquete sea enviado a nuestra casa.

En el parcial poner todas las entradas en orden, de mayor a menor.

No hay casos especiales, todos pasan por la misma tabla.

Subredes(VLSM)

Dada la red 192.168.80.0/24 se desea particionar en 4 redes:

- Guest - 14 hosts
- Robots - 58 hosts
- Servers - 29 hosts
- Workers - 120 hosts

Buscamos la cantidad de bits minima que necesitamos:

- 4
- 6
- 5
- 7

Arrancamos con la red que mas hosts tiene, entonces arrancamos con workers, mascara de red de workers:

11111111.11111111.11111111.10000000

255.255.255.128

Hay dos opciones, elegimos una para la red workers:

- **Network address:** 192.168.80.0
- **Usable host range:** 192.168.80.1 - 192.168.80.126
- **Broadcast address:** 192.178.80.127

Ahora, con los robots:

11111111.11111111.11111111.11000000

255.255.255.192

Ahora hay cuatro opciones, pero las primeras dos ya estan ocupadas por workers.

- **Network address:** 192.168.80.128
- **Usable host range:** 192.168.80.129 - 192.168.80.158
- **Broadcast address:** 192.178.80.159

Puede haber agarrado cualquiera de las demas, esta es solo una manera de resolverlo, en el examen mostrar todas las posibilidades.

Y asi sucesivamente con los que quedan.

Motivos para crear subredes

Puede haber razones topologicas:

- Host muy distantes
- Interconectar distintas capas fisicas
- Filtrar trafico entre redes

O puede ser por razones organizativas:

- Simplificar la administracion de la red
- Mapear la estructura de la organizacion
- Aislar el trafico

Superred(CIDR)

Mediante routers, hay varios motivos para usarlas:

- Optimizacion de enrutamiento
- Escalabilidad
- Mejor uso del espacio de direcciones
- Reduccion de latencia

Es gracias a que la IANA empezo a segmentar los RIR, las direcciones IP cobraron un orden jerarquico.

Paquete IPv4(datagrama)

Hlen no son 4 bits, sino que se debe multiplicar por 4, esto obliga a poner un relleno para que el length sea multiplo de 4.

Asignacion de direcciones IP

Como podemos asignar una direccion IP a una computadora? Puede ser:

- **Estatico:** Se asigna de forma manual
- **Dinamico:** Puede ser Local-Link(El dispositivo se asigna una IP a si mismo) o algun que otro protocolo(que se fueron deprecando): RARP, BOOTP y DHCP.

BOOTP era muy tedioso porque manualmente se tenia que configurar la IP de cada host.

DHCP

Dynamic Host Configuration Protocol, esta casi deprecado, se basa en UDP, por lo que transmite la informacion en binario. Lo que hace la computadora es enviar a la red un request pidiendo una direccion IP, si hay un servidor DHCP le va a responder con una IP asignada.

De parcial: Que protocolo tiene un puerto de origen cisco? DHCP

Son 4 paquetes(DORA):

- DHCP DISCOVER
- DHCP OFFER
- DHCP REQUEST
- DHCP ACK

Se usa el mismo **transaction id** durante la conversacion.

Si tenemos varias redes con un router y un servidor DHCP, entonces el router pasa a ser un DHCP relay. Cuando se llega al 50% del tiempo, la computadora pide renovar su IP con un **renew request**, el cual es respondido con un ACK del servidor. Ese era el tiempo T1(50%), si el servidor no responde, hay un tiempo T2 en el que la computadora vuelve a hacer DHCP DISCOVER.

Que ocurre si había un servidor que tenía una IP asignada? Va a cambiar su IP y habrá que avisar al servidor DNS del cambio. Hay un cambio adicional en el DHCP REQUEST, el código 81, que le pide al servidor DHCP que mande un DNS UPDATE al servidor DNS.

Network Address Translation(NAT)

(Ya lo anote en la unidad 6)

Que header modifica SNAT de el paquete IP? Modifica la dirección de origen.

DNAT(Destination NAT)

Se cambia no solo de IP pública a privada, sino que se cambia el puerto también. Esto hay que configurarlo en el firewall manualmente o se hace de manera automática mediante protocolos. Además, hay una tercera opción, se puede lograr una comunicación entre dos dispositivos mediante el uso de un “Relay”. En los routers de cada extremo se usa solo SNAT, pero entre medio se conectan al servidor de, por ejemplo, whatsapp, el cual se encargará de enviar la información al destino. Esto no está tan bueno pues gasta recursos de servidor entre medio, entonces se suele usar piping para eso, establece una comunicación point to point.

Existe UPnP(Universal Plug and Play), básicamente, si una aplicación necesita port forwarding simplemente usa este protocolo en vez de necesitar que un administrador lo haga manualmente.

Que necesita una interfaz para empezar a navegar? Dirección IP, máscara de red, dirección de gateway y direcciones de servidores DNS.

Holepunching se usa en cosas como llamadas de WhatsApp, para que en vez de tener que pasar por el servidor, los teléfonos obtienen la IP de cada uno mediante el servidor y utilizan SNAT para comunicarse entre sí directamente. El primer paquete de uno a otro se descarta pues no hay conexión todavía, pero se logra generar una entrada en el router. El que está del otro lado tiene que hacer lo mismo para crear las entradas y se establece la conexión.

ICMP

Esta encapsulado dentro de IP, es una discusión teórica si está en capa de red o de protocolo. Tiene sección de datos para hacer echo requests, así que se podría en teoría usarlo para transferir información, pero no conviene pues se bloquea(en ese caso es mejor usar DNS). Si mandamos un ping a loopback se está probando

todo lo que sea el funcionamiento interno del host. Si mandamos un ping a una computadora dentro de la red se esta probando que todo este bien conectado.

Utilidades básicas de networking

En caso de tener problemas de conectividad, el comando ping nos puede ayudar a localizar dónde se encuentra el problema.

Comando	¿Qué verifica?
<i>ping 127.0.0.1</i>	que la pila de networking esté funcionando y que los servicios de red estén habilitados
<i>ping dirección_IP_local</i>	que la placa de red esté configurada.
<i>ping host_en_mi_segmento</i>	que tanto la placa como el cable estén funcionando correctamente. Así también como el switch que los vincula y que el host remoto responde.
<i>ping host_en_otro_segmento</i>	que el router o la tabla de ruteo (dependiendo de la complejidad de la red) esté configurado correctamente
<i>ping host_fuera_de_la_red</i>	que los routers con salida hacia Internet o redes externas funcionen en conjunto con los routers de la organización si los hubiera

Tener en cuenta que muchos routers o hosts no responden solicitudes de eco

Figure 9: Diagnostico de red

IPv6

- Tienen un encabezado fijo(no hace falta padding)
- No permite fragmentacion(Era muy complicado)
- Estructura jerarquica de direcciones
- Permite la configuracion automatica
- Todos los dispositivos pueden tener una direccion exclusiva
- IPSec incorporado en el design(Antes era opcional)

Notacion abreviada

Hay reglas para poder escribir todo mucho mas rapido. Muchos ceros juntos → ::(solo una vez, sino no sabes cuantos faltan) Pero si hay mas de un grupo de ceros, entonces hay dos maneras de representarlos.

Tipos

- **Unique local address:** IP privadas, no pueden ser ruteadas a internet
 - **Local link address:** No pueden atravesar los router
- **Global address:** Internet

Clases

- 000: Unspecified, Loopback, Embedded IPv4 addresses
- 001: Global Unicast Addresses
- 010 - 110:Reserved by IETG for future use

- 111: Link-local, Unique-local, Multicast addresses

Tipos(de otra forma)

- Unicast
- Multicast
- Anycast

Anycast es para el caso donde hay varios hosts con la misma IP publica, permite conectarse al mas cercano que haya(pensando globalmente).

Interface ID

Son 64 bits, hay dos formas de determinarlo:

- **EUI**: Se basa en la direccion MAC
- **EFC 7217**: Genera direcciones de forma aleatoria, pero estables

La segunda opcion busca evitar que te puedan trackear, basicamente se podria determinar en que redes estas pues siempre vas a tener la misma direccion.

Flow Label ayuda al router con el ruteo, indica la ruta que hay que hacer, haciendo que lleguen mas ordenados y aliviando la carga de TCP.

Ipv\$-Mapped IPv6

Se puede mapear una IPv4 mediante IPv6, para que se pueda usar una direccion similar.

NAT64

Por si solas, las redes IPv6 no son compatibles con IPv4, para ello existe **NAT64** y **DNS64**.

Tunneling

Encapsular un protocolo en otro de **igual o mayor nivel**.

Que pasa si al querer comunicarse entre dos IPv6, hay una red IPv4 en el medio?

Se encapsula en IPv4, entonces habra dos headers IP mientras viaja en IPv4 y luego vuelve a ser IPv6.

En general se usa UDP si hay que encapsular pues tiene lo minimo necesario, hacer de wrapper.

Seguridad

En una empresa normal(IT) siempre hay que garantizar confiabilidad, lo menos importante es que esta andando el servicio. Caso contrario, si estamos en una empresa que se centra en el proceso pasa lo opuesto(hay procesos que no se pueden frenar). Sin embargo, en este ultimo caso no importa tanto si, por ejemplo, alguien se entera a que temperatura funciona un horno.

Entonces, en IPv4 tambien hay IPSec, pero es opcional. Tiene dos variantesL

- Modo transporte(entre dos hosts)

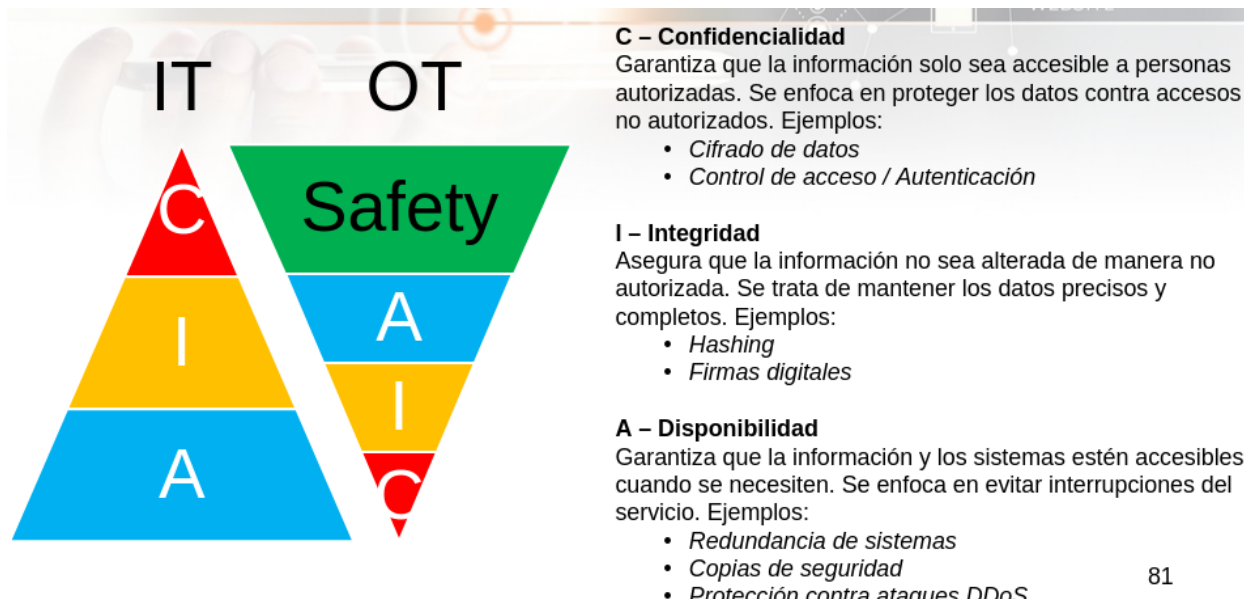


Figure 10: Seguridad

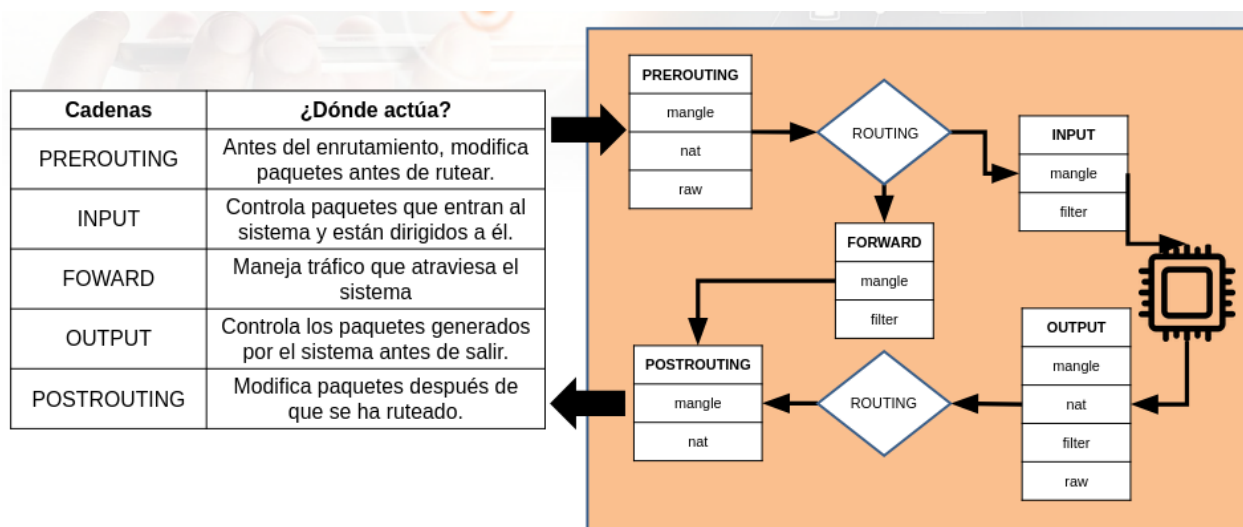
- Encapsulamiento(mas seguro, mas autenticacion y mas de un header)
- Modo tunel(VPN)

Firewalls

Antes solo se filtraba por IP y puerto, luego se empezo a implementar a nivel de circuitos(por si hacian como si ya tenian una conexion TCP establecida), luego subieron a nivel de aplicacion para validar la sintaxis de la conversacion, y finalmente se hace un filtrado dinamico, unieron circuit level con packet filter.

IP Tables

Despues de routear decide si aplicar NAT, por ejemplo, si va de una interfaz interna a otra interna, no hace falta cambiar la IP(si va de LAN a wifi).



Tablas	¿Dónde actúa?
filter	Filtrado de paquetes (permitir, denegar, registrar).
nat	Traducción de direcciones (DNAT, SNAT Masquerade).
mangle	Modificación avanzada de paquetes (ToS, TTL, marcado).
raw	Manipulación de paquetes antes del seguimiento de conexiones.

Hay algunos comandos basicos:

- **MASQUERADE**: Aplicar la IP publica como origen cuando sale para afuera.
- **SNAT**: Similar a **MASQUERADE** se puede ser lo mismo o tambien cosas extra como cambiar el puerto.

Solamente aplica esto al que inicia la conexion, no a todos los paquetes(checuar, pregunta importante).

Notas practica

- Para que sirve la capa de red?
- En vez de ser una conexion directa, se parte la informacion en paquetes, permitiendo que haya mas de una persona en la linea.
- En internet, si se corta un cable los paquetes se routean por otro lado, no se corta el servicio, igual si el cable esta saturado.
- Si UDP no responde, ICMP es el que se encarga de responder.
- En **route** usar **-n** porque no se va a entender nada sino.
- La capa de red no se encarga de ordenar los paquetes, se encarga transporte.
- Hay dos tipos de broadcast, uno a nivel red y otro a nivel enlace, la diferencia es como le llega a cada uno. Si haces un broadcast de capa 3, tiene que ser tambien de capa 2.
- Si tienes dos servidores DHCP en la misma red, los dos van a responder al DISCOVER y se va a elegir la respuesta que tenga mas puntos a favor, es decir, depende de la implementacion. Pero basicamente puedes dejar a todos sin internet si agarran el DHCP malo.
- Si el DHCP en una red es centralizado y hay muchos routers en el medio, se pone un **Relay agent** (puede o no ser el router, una compu). El agent hace de servidor DHCP, responde los DISCOVER, pero consultando antes al servidor DHCP.
- Los routers no propagan los broadcast.
- En el discover la mac tambien va con todos 1s: ff:ff:ff:ff:ff:ff
- En el OFFER, la IP destino antes era 255.255.255.255, pero ahora se puede especificar con un flag que se mande directamente la IP asignada, de manera que los demas lo pueden ignorar.

Unidad 8: Protocolos de ruteo

Sigue siendo parte de la capa de red. Hay varios tipos de ruteo:

- Ruteo minimo(trivial)
- Ruteo estatico(configurado manualmente)
- Ruteo dinamico(configurado automaticamente)

Ruteo minimo

En cada host hay una tabla de ruteo, por mas que se conecte directo a otra computadora, va a haber ruteo. Agarra una IP al azar cuando se prende el host.

Ruteo estatico

La tabla de ruteo es construida por un administrador en forma estatica. Sin embargo surge un problema, si hay que hacer un workaround o algo asi, hay que cambiar todas las tablas de ruteo de todos los routers.

Hay una columna mas en la tabla de ruteo, se llama metrica y se le puede asignar un peso para elegir.

Si la red es muy grande es muy complejo asignar pesos y configurar todo.

Ruteo dinamico

La tabla de ruteo es construida en base a informacion intercambiada por protocolos de routing. De manera que si hay algun corte el algoritmo se encargara de volver a balancear las tablas de ruteo. La **convergencia** es el tiempo que se tarda en balancear las tablas de ruteo, se busca que el algoritmo converga lo mas rapido posible. Hay varios tipos:

Los protocolos pueden ser IGP o EGP, internos o externos a una organizacion respectivamente.

Algoritmos de ruteo descentralizado

Los routers solo se comunican con su vecino para construir las tablas. **Distance vector algorithm**, pero cuando hay un corte de enlace o hay despasaje de tiempo se propaga y se rompe. Se quedan contando entre si y paran cuando se llega a infinito y sale el problema de definir infinito. Tiene un par de workaround, el mas sencillo es **spli-horizont**, tambien esta **reverse...(no lo entendi)** y **hold-down-timer**(desconfia de los demas por un cierto tiempo, tiempo de hold mayor que el de actualizacion).

- RIP(IGP)
- BGP(EGP)

Algoritmos de ruteo globales

Link state algorithm, usa grafos y primero propaga un paquete de manera de inundacion, y esto tiene como resultado que si hay muchos routers se saturan los routers y se empiezan a tirar paquetes. Luego, usa djikstra para determinar el mejor camino.

- OSPF(IGP)
- IS-IS(IGP)

Sistemas autonomos

Conjunto de redes bajo una misma administracion o politica de routing comun. Utiliza algoritmos de routing para comunicarse internamente(IGP) y externamente(EGP).

RIP

Routing Information Protocol, la version 1 era muy naive, no tenia autenticacion, notacion classful y no soporta CIDR/VLSM. Mientras que la version 2 mejoro bastante y era classless(no determinada la mascara en base a la clase). Fue ampliamente usado porque venia preinstalado en BSD.

OSPF

Open Shortest Path First, soluciona el problema de la saturacion separando en areas. Entonces solo pueden hacer flooding en su area. Hay un backbone que recorre toda la red y se divide esa misma red. Entonces, todas las areas de OSPF son un sistema autonomo. Los routers entre medio toman un valor general de un area para pasarle a la otra.

Unidad 9: Capa de enlace

Cumple con un servicio basico, mover paquetes desde un nodo al nodo adyacente sobre un enlace de comunicacion. Tiene varias propiedades:

- Encapsulamiento
- Acceso al medio
- Comunicacion confiable
- Control de flujo
- Deteccion de errores
- Correccion de errores
- Half-duplex o full-suplex

Existen dos tipos de comunicaciones a nivel enlace:

- Broadcast(LAN)
- Point to point(WAN)

Point to point

Los protocolos de nivel de enlace de este tipo son mas simples que para enlaces broadcast:

- Generalmente usados para encapsular datagramas IP
- No se necesita control de acceso al medio

Protocolos

- SLIP(simple)
- PPP(sucesor de SLIP)

Broadcast

Es un bus, un canal compartido. Tiene posibilidad de interferencia, lo que significa que puede haber colisiones (dos signals al mismo tiempo). Entonces, debe existir una norma de como de comparte el canal, la coordinacion se hace usando el mismo canal de datos.

Protocolos

Son varios protocolos de acceso simple que pueden ser:

- Protocolos de acceso aleatorio (ALOHA, CSMA, CSMA/CD, CSMA/CA)
- Protocolos de acceso controlado (Reservation, Polling, Token passing)
- Protocolos de acceso canalizado (FDMA, TDMA, CDMA)

Los protocolos de acceso canalizado se pueden dividir for frecuencia, por tiempo o por codigo. Los de acceso controlado como token ring usan un timeout para recuperar el token si un host se queda trabado. Los de acceso canalizado son los mas interesantes, cuando un nodo quiere transmitir: transmite. Si dos nodos transmiten al mismo tiempo hay una colision, el protocolo a usar debe definir:

- Como detectar colisiones
- Como actuar frente a las colisiones

Este ultimo tiene en juego el tiempo de propagacion de los electrones, pueden ocurrir colisiones si uno transmite, pero no escucho otro que habia empezado a trasmitir. CSMA usa un tiempo aleatorio de reintento, cada vez prueba uno mas grande hasta que pueda transmitir. Una variacion es CSMA/CD (Collision detection), cuando uno detecta colision, hace un broadcast a toda la red anunciando una colision.

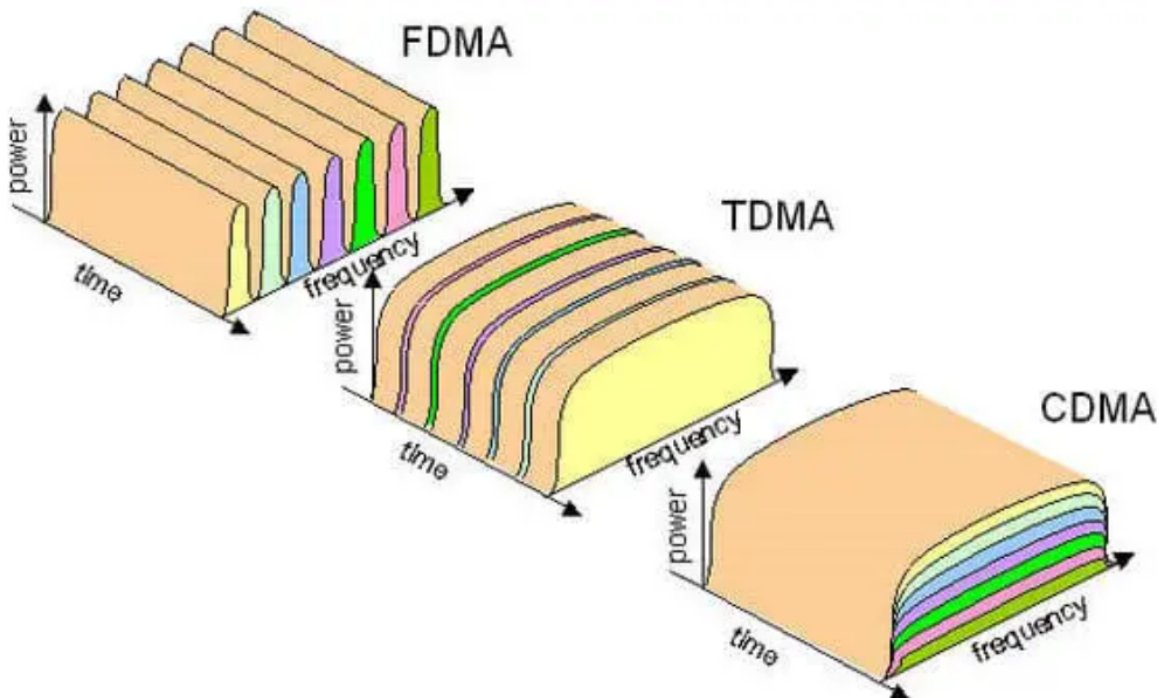


Figure 11: Protocolos de acceso aleatorio

Las de acceso canalizado usan codigos ortogonales o transformadas de fourier.

Ethernet

Topologia bus

Surcio de manera muy rustica, era muy economico y se basaba en un cable coaxial. Todos los host estaban en la misma linea, conectados a un hub. Cada host se puede conectar.

Topologia estrella

Cada computadora se conoce a un switch, cada host se puede conectar y desconectar sin afectar al resto de la red, se basan en el MAC address y se deben evitar ciclos.

MAC address

Seis octetos, tiene varios bits para indicar si es Unicast o Multicast, y si es Universally administered addresses o no.

Preamble

Son 7 octetos, usados para sincronizar la transmision.

ARP

La direccion MAC es conocida, pero no siempre lo sabe el otro host, entonces se crea un paquete preguntando quien tiene tal IP y sera respondido con la IP y la MAC address. Si el otro host esta en otra red, hay una direccion de la gateway, entonces, obtiene la MAC del gateway.

Similarmente a ICMP(aunque es entre red y transporte), no es de red(no conecta entre hosts) ni de enlace(no transmite datos de un host a otro).

Se pueden poner dos maquinas con la misma MAC address con IP distintas, una puede ser dinamica y la otra estatica. Lo que haga con el paquete depende, de forwarding y otros.

Este protocolo se utiliza de distintas formas, por ejemplo, RARP

Si hay dos redes y una de ellas es oculta, entonces, la mac de respuesta es la del router.

ARP spoofing

ARP no contempla autenticacion, cualquiera puede responder un ARP request para relacionar un IP con un MAC, lo que permite distintos tipos de ataque:

- Man-in-the-middle
- Denial-of-service
- Session hijacking

Algunos programas para experimentar: Arpspoof, Arpoison, Cain & Abel, Ettercap

Hay varias maneras de mitigar esto, pero la mas sencilla es pasar a IPv6, que usa la MAC en la misma IP address para identificar los host.

Notas practica

- En CSMA-CD se definen restricciones sobre el size maximo de la trama y el largo del cable(trama el doblde del minimo). Entonces no va a haber colisiones no detectables(esto puede ocurrir si una maquina termina y luego ocurre la colision).
- Los routers ayudan a separar las redes en dominios de colision(redes broadcast).
- Un switch es como un router(en caso de ser un switch de capa 3), pero con mas bocas. **No** se puede tener un loop de switched, sino ocurre una tormenta de broadcast.
- Hay protocolos que permiten que haya loops de switchs.
- IP sirve para conectar distintas redes, cosa que no se podia con ethernet.
- Trama es capa de enlace, paquete es IP.
- Con ARP, cuando mandas broadcast, todos pasan a tener tu IP y MAC.
- En IPv6 no hay ARP.